

Different OOAD Models

1. Class Model
2. Use Case Model
3. Interaction Model
4. Object Model
5. State Model
6. Activity Model
7. Component Model
8. Deployment Model
9. Package Model

Why So Many Models?

Each model serves a specific purpose and perspective:

- Define what the system is made of.
- Describe how the system behaves.
- Detail how the system is built and deployed.

The Three Models

1. Logical Model (NPTEL):

- Represents the conceptual structure of the system, focusing on what the system does without considering implementation details. It includes entities, their attributes, and relationships in a technology-agnostic way.
- In OOAD, this often corresponds to the **Class Model**, as it defines the logical entities (classes) and their relationships.
- Example: A class diagram showing Customer, Order, and their associations in an online shopping system.

2. Physical Model (NPTEL):

- Describes how the system is implemented in terms of hardware, software components, or deployment. It focuses on the physical or technical aspects, such as databases, servers, or component distribution.
- In OOAD, this might map to **Component Diagrams** or **Deployment Diagrams**, which detail the physical architecture or implementation-specific structures.
- Example: A deployment diagram showing how the shopping system's database and application server are hosted.

3. Behavioral Model (NPTEL):

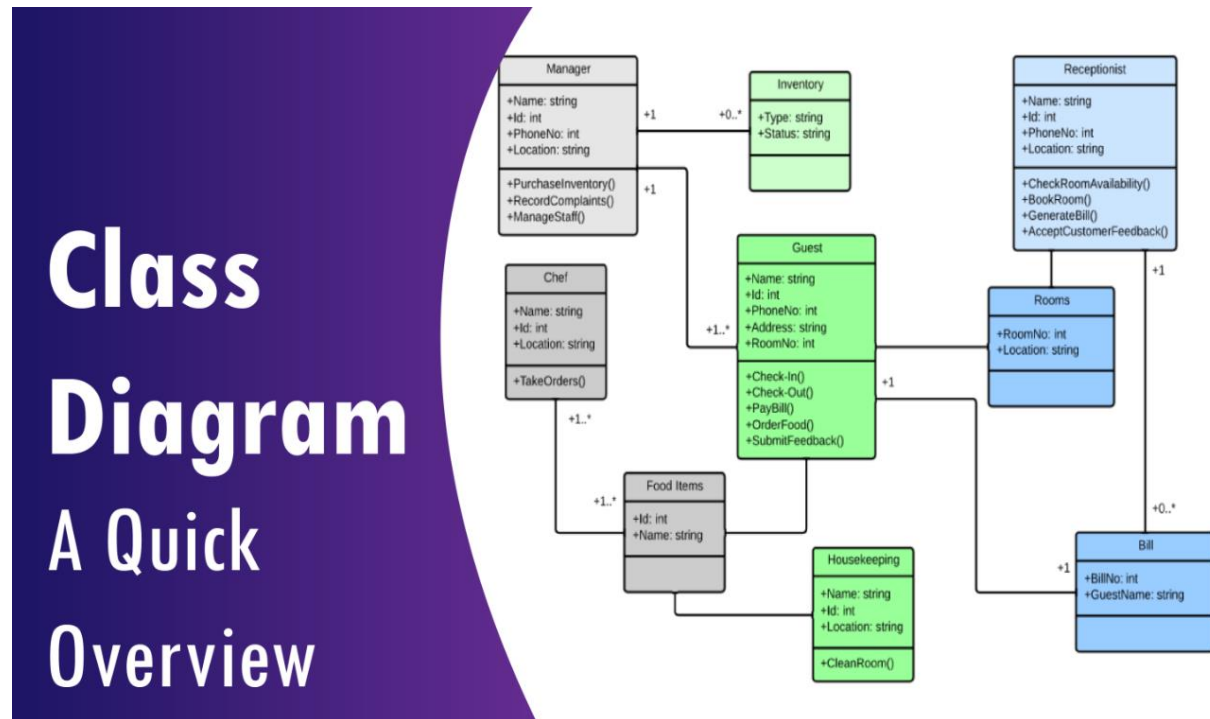
- Captures the dynamic behavior of the system, showing how objects interact or how processes flow over time to achieve system functionality.
- This aligns with the **Interaction Model** (e.g., Sequence Diagrams) and other behavioral diagrams like Activity Diagrams or State Machine Diagrams in OOAD.
- Example: A sequence diagram showing the interaction between a Customer and PaymentGateway during checkout.

Class Modeling

What is Class Modeling?

Class modeling is a fundamental activity in OOAD that focuses on creating a static blueprint of a system by identifying and defining classes (abstractions

of real-world entities) and their relationships. It aims to capture the structure of the system, including the data (attributes) and behaviors (methods) of objects, as well as how classes interact with each other.



Key Components of Class Modeling

1. Classes:

- A class represents a blueprint for objects with similar properties and behaviors. For example, in an online shopping system, Customer, Order, and Product are classes.
- Each class has:
 - ❖ **Attributes:** Properties or data (e.g., customerId, orderDate).
 - ❖ **Operations/Methods:** Behaviors or functions (e.g., placeOrder(), calculateTotal()).

2. Relationships

- **Association:** A general relationship between classes (e.g., a Customer places an Order).

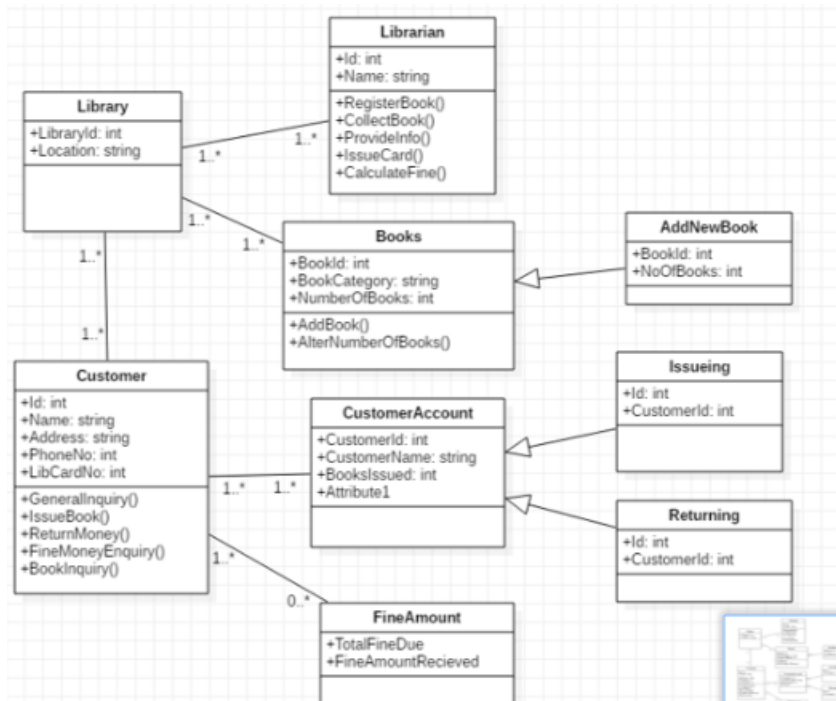
- **Aggregation:** A "whole-part" relationship where one class is a container (e.g., an Order contains OrderItems).
- **Composition:** A stronger form of aggregation where the part cannot exist without the whole (e.g., OrderItems are destroyed if the Order is deleted).
- **Inheritance:** A relationship where one class (subclass) inherits attributes and methods from another (superclass), e.g., PremiumCustomer inherits from Customer.
- **Dependency:** A weaker relationship where one class uses another temporarily (e.g., a PaymentProcessor depends on a BankService).

3. Multiplicity

- Specifies how many instances of one class can relate to another (e.g., one Customer can have many Orders—a 1-to-many relationship).

4. Constraints and Rules:

- Additional rules or conditions (e.g., an Order must have at least one OrderItem).



Steps in Class Modeling

1. Identify Classes:

- Analyze the problem domain (requirements, use cases, or user stories) to identify entities. For example, in a library system, classes might include Book, Member, and Loan.
- Use noun-phrase analysis: Nouns in requirements often become classes, while verbs indicate operations or relationships.

2. Define Attributes and Operations

- Determine the data each class holds (e.g., Book has title, author, ISBN).
- Identify behaviors (e.g., Book has borrow(), return()).

3. Establish Relationships:

- Identify how classes interact (e.g., a Member borrows a Book, creating an association).
- Specify relationship types (association, aggregation, etc.) and multiplicity.

4. Refine the Model:

- Eliminate redundant or unnecessary classes.
- Ensure the model aligns with system requirements and object-oriented principles (e.g., encapsulation, single responsibility).

5. Validate the Model

- Review with stakeholders to ensure the model accurately represents the system.
- Use the model to simulate scenarios (e.g., how a Member borrows a Book).

6. Document in a Class Diagram:

- Represent the classes, attributes, operations, and relationships visually using UML notation.

Benefits of Class Modeling

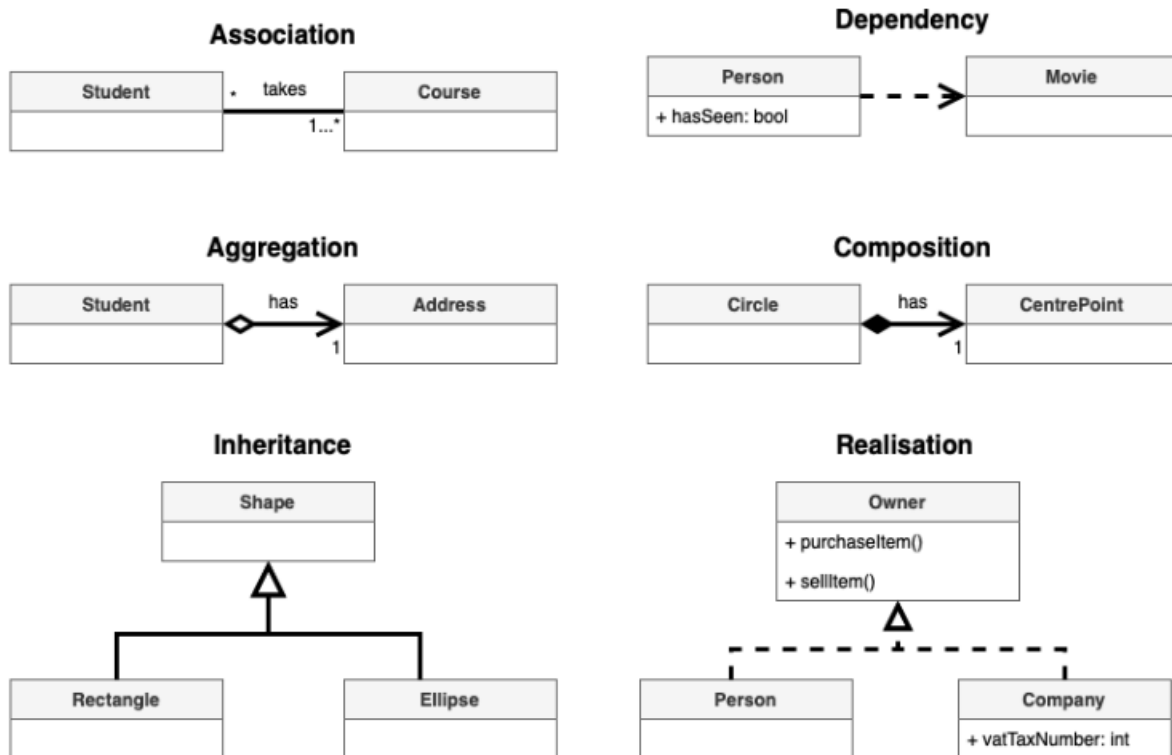
- **Clarity:** Provides a clear, visual representation of the system's structure.
- **Modularity:** Encourages reusable and maintainable code through well-defined classes.
- **Scalability:** Makes it easier to extend the system by adding new classes or modifying relationships.
- **Error Reduction:** Identifies design flaws early, before coding begins.

A Sample Class Model:

UML Class Diagram Notation:

A class diagram, as the output of class modeling, uses standard UML symbols:

- Class: A rectangle with three sections (name, attributes, operations).
- Relationships:
 1. Association: A solid line between classes (e.g., Customer — Order).
 2. Aggregation: A line with an empty diamond (e.g., Order ◇— OrderItem).
 3. Composition: A line with a filled diamond (e.g., Order ◆— OrderItem).
 4. Inheritance: A line with a hollow triangle pointing to the superclass.
 5. Multiplicity: Numbers like 1, 0..*, or 1..* to indicate cardinality.



Example: Class Modeling for an Online Shopping System

1. Identify Classes

- Customer, Order, Product, ShoppingCart, Payment.

2. Define Attributes and Operations:

- Customer: Attributes (customerID, name, email); Operations (placeOrder(), viewOrderHistory()).
- Order: Attributes (orderID, orderDate, totalAmount); Operations (addItem(), cancelOrder()).

3. Establish Relationships

- Customer has a one-to-many association with Order (one customer can place many orders).
- Order has a composition relationship with OrderItem (items are part of an order).
- ShoppingCart aggregates Product (cart contains products).

4. Create Class Diagram:

- Draw classes with attributes and operations.
- Connect them with appropriate relationship lines and multiplicity (e.g., 1 Customer to 0..* Orders).